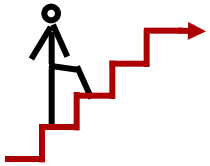


Working with Databases using JDBC



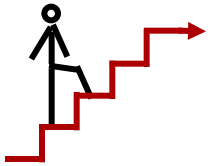
Transaction Processing

Transaction:

A transaction is a set of one or more operations that together form a single logical unit of work.

Example:
1. Deduct ₹1000 from ABC's account.
2. Add ₹1000 to XYZ's account.

These two operations must succeed or fail together — not halfway.



Transaction Processing

A – Atomicity (“All or Nothing”)

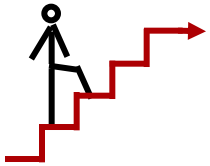
A transaction is atomic — it either completes fully or doesn’t happen at all.

Think of an online shopping payment — either:

- **The money is deducted, and the order is placed, or**
- **Nothing happens (no money lost, no order created).**
- **No “half-paid” state is allowed!**

•In short:

- **Atomicity = “Do everything or do nothing.”**



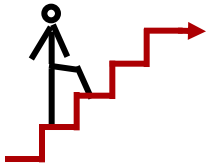
Transaction Processing

C – Consistency ("Valid State → Valid State")

A transaction must bring the database from one consistent state to another — following all rules, constraints, and relationships.

Imagine a game score table: total points must always equal the sum of individual scores. If one update breaks that rule, database becomes inconsistent.

- **In short:**
  - **Consistency = "The rules of the database are never broken."**



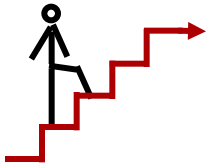
Transaction Processing

I – Isolation (“No Mixing of Transactions”)

Even if many transactions run at the same time, they should not affect each other’s intermediate states.

Two people editing the same Google Doc at once — each should see a stable version until they’re done, not each other’s half-typed text!

- **In short:**
  - **Isolation = “Transactions act as if they run alone.”**



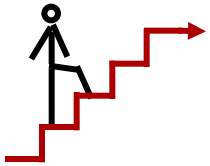
Transaction Processing

D – Durability (“Permanent Once Done”)

Once a transaction is committed, its effect is permanent — even if system crashes or restarts.

Clicking *Save* on a document — even if your laptop restarts, the saved content remains.

- **In short:**
  - **Durability = “Once done, it stays done.”**

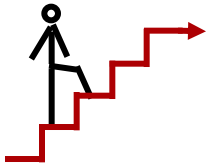


Transaction Processing

Use the following three methods of Connection interface to handle transaction Processing:

- public void setAutoCommit(boolean)
- public void commit()
- public void rollback()

- public boolean getAutoCommit()



Transaction Processing

```
try {
    conn.setAutoCommit(false);
    debitAccountA(500);
    creditAccountB(500);
    conn.commit(); // All OK
} catch (Exception e) {
    conn.rollback(); // Undo all if error
}
```